Rldcoin: Interstellar Peer-to-Peer Payments

Runlai Deng
dengrunlai@gmail.com
www.rldcoin.com
19 October 2026

Abstract. A community separated from another by years of communication delay needs money whose local history does not depend on a later contest with distant computation. We propose a payment architecture for one native currency, runlai (RLD), in which each spendable output has one authoritative home domain. A domain finalizes local transfers under an explicitly accepted Byzantine fault model. To pay another domain, the owner first removes value from local spendability and obtains a recipient-bound export certificate. The destination verifies that certificate and atomically records either one import or one permanent rejection. An import creates local spending authority; a rejection can authorize recovery at the source. Silence authorizes neither. This separation permits one-way delivery, independent local payments during interstellar disconnection, and onward spending without a return message. We give the payment and issuance transitions, a conservation argument over causal histories, and the conditions under which unrelated reconnection cannot reverse a local payment. A fixed budget of twenty-one million RLD is divided into conserved issuance strands that can be relocated before isolation; public work consumes a strand's next entitlement without selecting a global payment history. The cost is explicit: users must accept the security of the domains in their funds' provenance, a failed quorum can stop its domain, and an unreachable destination can strand exported value. The architecture combines local settlement and asynchronous authority transfer without a global transaction order or compulsory central relay. RLD is the sole payment and fee currency throughout the system; crossing domains transfers existing RLD rather than issuing another currency. It makes a specific claim about conserved payment under stated trust assumptions, not an unconditional claim about future adoption, physical delivery or cryptographic permanence.

1. A payment system for separated worlds

Consider two settlements that can communicate only through messages taking years to arrive. People at either settlement still need to pay wages, buy food and transfer savings. Requiring both settlements to approve each purchase would make their communication delay the minimum payment time. Letting both independently spend the same funds would create a different problem: the system could deliver two apparently valid claims against one value.

A single greatest-work history resolves competing branches by eventually discarding one. That is useful under its network and adversary assumptions [1], but it is a poor basis for the promise that an isolated settlement's years of ordinary commerce will remain part of its monetary history. Even honest communities can grow incompatible branches while contact is absent. More local confirmations do not reveal an unseen remote history.

We instead locate the authority to resolve each spend conflict before allowing the spend. Value usable in domain A is not simultaneously usable in domain B. Within A, the local settlement protocol decides whether an owner has spent an output. A payment to B first makes the output unusable in A, then carries evidence allowing B to exercise the transferred authority once. A transport receipt is not sufficient; a certified monetary transition is required.

This is an allocation of authority, not a way around propagation delay. Local payment can finish without a distant message because its inputs already have a local home. Newly transferred authority still needs a real sequence of contacts to reach its destination. A community that wants independent operation must place its funds and sufficient validation capacity there before losing contact.

The intended benefit is durable economic separation: losing a route to another world need not erase an already settled local wage or require a distant institution to approve tomorrow's purchase. The same mechanism is relevant to remote

communities, disrupted infrastructure and intermittently connected vehicles. Its usefulness depends on the security and availability assumptions below, rather than on geographical labels.

2. Model, identities and assumptions

A domain is a replicated state machine with a stable identity D, an authenticated initial descriptor and a sequence of certified configurations. It need not coincide with a planet. Two nearby communities may use different domains, while a badly chosen domain spanning a stellar gap would inherit that gap's local confirmation delay. Domain creation is open; acceptance of a domain's certificates is an explicit user and receiving-domain decision.

The baseline domain has n = 3f + 1 distinct validators and a finalization rule requiring a protocol-specific commit proof from at least 2f + 1 validators. It uses an established Byzantine state-machine-replication protocol with its locking and view-change rules, such as the family represented by PBFT and HotStuff [2,3]. A list of signatures on an arbitrary state root is not a replacement for that protocol's commit proof. Honest validators execute every state transition, preserve durable consensus locks and do not finalize conflicting states at the same position.

Safety assumes no more than f Byzantine validators in each relevant configuration, unforgeable authorization and certificate signatures, collision-resistant commitments, correct verification, and recovery that does not erase committed state or import decisions. Liveness additionally requires a responsive local quorum, eventual local communication conditions sufficient for the selected consensus protocol, fair admission and adequate resources. Safety does not require a timely interstellar return path. Eventual delivery does require an eventual usable forward path.

Owners control spending keys independently of validators and relays. An adversary may copy an owner's key, submit conflicting authorizations, control carriers, duplicate or reorder messages, delay traffic indefinitely, and advertise false routes. It cannot exceed a stated validator fault threshold or break a cryptographic assumption within a claim that relies on that assumption. A threshold compromise is a security failure, not a recoverable network delay.

A client authenticates an initial domain descriptor through a separately accepted trust anchor. A peer advertisement, hostname or self-signed new descriptor cannot establish that anchor. The descriptor commits to the protocol and common RLD monetary rules, validator identities, signature and hash suites, epoch-transition rule, fee policy and finite resource limits. Distinct identities do not establish independent operators. Admission and validator replacement are governance and Sybil-resistance decisions; the payment protocol does not turn arbitrary public keys into trusted validators.

The claims concern digital ledger rights. RLD is a native digital currency, not a certificate of an external reserve. Its purchasing power and exchange rate are not protocol guarantees. Nothing in a payment certificate proves delivery of physical goods or a corresponding bank deposit.

3. Money with one authoritative home

The sole native payment currency is runlai, identified by the ticker RLD. One RLD equals 10^24 atomic runlai units. Every ledger amount is a bounded nonnegative integer in these atomic units; floating-point conversion and rounding cannot authorize value. Local payments, interstellar principal, source and destination fees, recovery fees and protocol relay-service payments all use this same currency. Another token, a domain-issued substitute or a wrapped claim cannot discharge an RLD payment within this protocol.

A common RLD currency descriptor commits to the monetary genesis, denomination, initial supply, complete issuance and retirement rules, and their authorization conditions. Its digest is the currency identity C_RLD. Every domain, output, invoice and ticket binds that exact identity. A matching name or ticker is insufficient: an unrelated descriptor with another genesis or monetary rule is a different currency and is refused. Authenticated protocol upgrades must preserve the same monetary history and cannot create a second RLD balance by changing an identifier.

The maximum lifetime issuance is C = 21,000,000 RLD = 21 x 10^30 atomic runlai units. Genesis creates zero spendable RLD and no founder, investor, treasury or validator allocation. All C units are reserved for the public work mechanism below. The cap and denomination are monetary policy choices, not estimates of an interstellar economy's required market value; changing the unit count alone creates no purchasing power. New domains start with zero RLD and no issuance

entitlement. Transfers relocate already issued RLD, while a separately typed issuance strand represents a bounded ability to issue in the future. A strand is not a spendable RLD output or another native currency.

A spendable output records its identifier, C_RLD, integer amount, owner authorization policy, home domain and provenance set. Only the home domain can consume it. A local transfer consumes unspent inputs and produces new outputs, with any fee represented as an ordinary RLD output to the domain's declared service rule. Every required actual input owner authorizes the complete operation. Inputs must bind C_RLD and share one home; cross-domain multi-input atomic purchases are outside the basic transfer operation.

```
LocalTransfer:
  require all inputs and outputs bind C_RLD
  consume existing, authorized, unspent local inputs
  create recipient, change and service-fee outputs
  require sum(inputs) = sum(outputs)
```

Geographical movement of a key does not move its output's home. A traveler who takes only a key into an isolated ship cannot spend an Earth-homed output there without contacting Earth's authority. The traveler must relocate authority before departure or accept a ship's separately offered credit. Credit is a distinct liability, never a silently duplicated balance.

The basic state contains an unspent-output map U, a source export map E, a destination decision map J, an authenticated configuration history and resource/fee rules. At a destination, J[x] is initially absent and can become Imported or Rejected. At a source, E[x] records Exported and can later record Refunded following authenticated rejection. Receipts may annotate an export as delivered, but do not restore local spending authority.

3.1. A finite issuance budget

Genesis commits to exactly N = 4096 distinct issuance strands, numbered s = 0 through 4095. Each has L = 1,000,000 successive work slots and the same reward rho = 0.005126953125 RLD = 5126953125000000000 atomic units. Thus the product of N, L and rho equals C exactly, without rounding or a residual reserve. Every accepted slot pays its entire reward to its work author's bound payout policy. There is no separate founder percentage, validator subsidy, sale allocation, staking return or tail emission. Fees for ordinary payments are transfers of existing RLD, not additions to this budget.

A strand records its currency, number s, consumed-slot count k, one authoritative home, current work anchor and monetary provenance. Its unissued budget is (L-k) rho for 0 <= k <= L. The identity of the next issuance event is H("RLD-MINT", C_RLD, s, k); changing home, validator keys, epoch or payout address cannot reset k or create a new strand. Strands cannot split, merge, renew or multiply. An exhausted strand authorizes no further reward. The finite strand count limits issuance concurrency, not the number of payment domains or users.

All strands initially reside as public protocol state in the one issuance-bootstrap domain D_0 named by genesis. They have no private spending owner. D_0 uses the same authenticated BFT rules as other domains; it is a declared initial authority, not proof of globally permissionless consensus. Its current quorum can relocate whole strands through the procedure below, but cannot change the reward, cap, work rule or number of remaining slots, and cannot pay the reserve directly to any account. No domain is entitled to a new strand merely because it exists or claims a geographical name.

This bootstrap deliberately concentrates initial allocation of issuance capacity. Selecting D_0 and its operators is a visible trust decision. Strand placement is public discretionary governance by the current home quorum, not a claimed identity-free fair lottery among communities. A quorum can favor destinations or refuse relocation; its signatures do not prove demographic fairness. Once an authenticated move completes, the new home can issue from that strand without D_0 or any other remote domain remaining online. Existing local RLD payments require neither a strand nor an active mining process.

3.2. Public work and a mint transition

Anyone with an owner's signing policy and access to a strand's home may compete for its next slot, without a protocol account, existing RLD balance, stake or compulsory pool. The home admits work under bounded resource and fair-processing assumptions; admission by an accepted validator domain is still required. The first valid candidate finalized for a slot receives rho; a valid proof that loses the local ordering race receives nothing. More identities create no additional slot or fixed identity reward. Computation is used to distribute new RLD, not to grant consensus votes or reorganize settled payments.

A strand's anchor a_s is a SHA256 commitment to the tagged opening statement of its current slot: C_RLD, s, k, home and either the genesis-opening tag, exact preceding Mint body hash, or completed installation/recovery decision identifier and position. A Mint body contains its old anchor, candidate and payout; it excludes the resulting state root and next anchor, avoiding a circular commitment. Ordinary payment blocks and authorized validator replacement leave the anchor unchanged. Mint finalization closes the old slot and opens the next one. A move suspends the source slot; import or certified recovery opens it with a new anchor at the receiving home. Candidates bind the home and anchor, so a proof for an earlier location or closed slot cannot be transplanted.

The work profile uses Argon2id version 0x13 as described by RFC 9106 [15], with m = 65536 KiB, t = 3 passes, p = 1 lane and a 32-byte result. Secret and associated-data inputs are empty. Define SHA256 as a 32-byte digest and enc as the unambiguous typed canonical encoding used for signed protocol objects. The candidate includes the exact payout-policy commitment o and a 32-byte nonce n. Its work inputs and fixed target are:

```
w = enc("RLD-WORK-1", C_RLD, s, k, home, a_s, o, n)
P = SHA256(w)
S = first16(SHA256(enc("RLD-WORK-SALT-1",
                       C_RLD, s, k, home, a_s)))
y = Argon2id(P, S; m=65536, t=3, p=1, v=0x13, out=32)
require uint256_big_endian(y) <= 2^240 - 1
```

The owner authorizes the exact Mint candidate, full payout policy and these bytes. Copying a candidate cannot change its payee. Under a uniform-result model the target succeeds with probability 2^-16 per evaluation, or 65,536 evaluations on average; this is a work estimate, not a time deadline. The specified one-lane computation fits general-purpose memory, but neither RFC 9106 nor this selection proves ASIC resistance, equal device efficiency, profitability or useful-work output. There is no automatic difficulty retarget and no reward halving. These parameters define a concrete baseline work profile, not a measured hardware-economic optimum.

```
Mint(s, k, home, a_s, payout_policy, nonce, owner_auth):
  require strand s is active here and k < L
  require exact stored k, home, a_s and C_RLD
  require valid owner_auth and the bound work result
  atomically, under local consensus:
    strand.k = k + 1
    create one RLD output of exactly rho for payout_policy
    retain exact issuance event and advance strand anchor
```

The new output inherits the strand's full authority provenance and the executing configuration. The work author receives 100% of rho, with no deduction from the unissued reserve for validators or relays. Mint admission has no required RLD payment, permitting zero-balance participation; operators still need separately funded resources and may refuse excess load before custody. Incorrect or duplicate candidates cannot consume a slot. A proposal, solved hash or preliminary vote is not issued money. The reward becomes spendable only when the complete Mint transition and output are durably finalized under the local quorum assumptions. No remote acknowledgment or probabilistic mining maturity follows it.

A home cannot finalize both Mint and relocation of the same active strand state. During an intra-domain partition, the ordinary quorum rules still apply. A single surviving miner or validator cannot create a replacement authority. The current work profile is bound by the common monetary descriptor; ordinary domain governance cannot lower its target, change parameters or bypass it. This baseline has no Mint transition under another work profile. If confidence in the work primitive fails, affected issuance stops. A proposed cryptographic replacement needs a separately specified and

authenticated successor monetary protocol that preserves counters and the cap and prevents old/new double exercise; no such amendment is authorized by ordinary local fee or validator governance.

3.3. Moving issuance capacity between worlds

A community wanting independent future issuance must obtain an active strand before losing the relevant contact path. It can receive a whole strand from any current home, not only D_0. A destination quorum first signs an authenticated readiness request binding its stable identity, currency and supported strand rules. The current home may then finalize a MoveStrand governance transition with its ordinary commit proof; no founder signature or private reserve-owner key is sufficient. This authority controls placement of future public work, not a right to receive its rewards.

The move ticket has kind RLD_ISSUANCE_STRAND and binds C_RLD, s, k, remaining budget, old anchor, complete provenance, source, destination, unique move identifier and recovery rule. Every mint and move identifier includes its action kind; payment and strand decision namespaces are distinct. Source finalization removes the active strand before releasing the ticket. The destination authenticates the complete lineage from genesis and atomically chooses one installation or one permanent rejection. Installation creates one active strand with the same s and k, adds the required authority provenance and opens a new destination work anchor. It creates no spendable RLD and consumes no slot.

Only authentic permanent destination rejection can restore that exact strand at the source, once, with a new source work anchor. Recovery retains both domains' relevant provenance. The payment protocol's import-or-reject exclusion and no-timeout reasoning apply to this separately typed object; partial movement, counter reduction, mixing payment credit with installation, and fallback to another destination are invalid. Old home copies cannot mine after source finalization. A new destination cannot import an already installed or rejected move, even after another move takes the strand onward. Full lineage, permanent decisions and continuity survive state compaction and authorized succession.

A move pays no fee out of unissued capacity. Any separately agreed relocation service fee is an ordinary funded RLD payment and cannot alter remaining budget, slot reward or ticket identity. The parties may sponsor the transport and validation cost. If either the required quorum or complete evidence is permanently lost before installation or certified recovery, the remaining budget can stay unusable. There is no emergency reallocation of apparently missing strands and no remote timeout that reissues them. This loss reduces attainable supply; it never enlarges the cap or unwinds unrelated settled payments.

3.4. Release pace and sustainable operation

There are exactly N times L = 4,096,000,000 reward events available. If all are validly finalized, gross lifetime issuance reaches exactly C. If work, quorum availability or a strand's recovery fails, actual issuance can be lower. The total finalized-event rate lambda gives gross issuance rate rho times lambda before exhaustion. No fixed number of years to completion, synchronized stellar clock, per-planet annual quota or late-arrival catch-up reward is defined. More effective computation can exhaust slots faster; faster hardware and wealth can concentrate rewards. The cap establishes scarcity of valid units, not price, fair individual income or a calendar supply curve.

An available but non-mining domain can continue settling its existing RLD and earn fees for actual service. A remote domain with no strand has the same payment capability but no right to mint. After all slots are consumed, newly issued rewards stop permanently; validators and carriers rely on explicit RLD fees, voluntary funding or other disclosed operating support. Empty self-payments, relay receipts, uptime, newly created identities and administrative declarations create no subsidy. A fee policy alone does not prove a sufficient long-term security budget.

Retirement is an owner-authorized destruction of existing RLD and is permanent. Lost keys do not prove destruction. Neither retirement nor presumed loss replenishes a strand, reduces k or authorizes replacement money. Fixed supply and finite precision may create long-term liquidity and loss risks; this design does not hide them behind automatic inflation. The choice favors a verifiable lifetime limit and local payment autonomy over guaranteed perpetual issuance at every new settlement.

4. Local settlement and its meaning

An owner signs C_RLD and the exact domain identities, operation type, input references, ordered output policies and amounts, explicit fees, and a nonce or input-derived uniqueness commitment. Distinct action tags separate a local transfer, Mint, MoveStrand, export, rejection request and recovery. Canonical encoding distinguishes field types, lengths and integer values. Hashes and signatures bind the complete object, including protocol and cryptographic-suite versions. The baseline can use established ordinary signatures such as Ed25519 [4]; its long-term compromise implications are considered in Section 13.

A node first verifies the owner authorization, input availability, amounts, provenance and admission conditions. The local consensus engine orders competing uses of an input and finalizes the resulting state. A client reports a locally settled payment only after checking the commit evidence and its exact output inclusion, and retaining recoverable receipt material. A submission acknowledgment or a preliminary quorum vote is not settlement.

A recipient can consume a settled output in the next local state transition. No separate global block, mining reward maturity or stellar acknowledgment is required. Connected participants can therefore make low-latency payments directly on their local final ledger. The design does not need an illustrative payment channel to explain ordinary local use.

This finality is conditional on the domain and provenance fault assumptions. It means that valid future protocol execution does not reorganize an already finalized local prefix. It does not mean that validators cannot censor, that keys cannot be stolen, that software cannot fail, or that a compromised quorum has a cryptographic obligation to repair losses. A damaged domain has an incident requiring explicit recovery and acceptance decisions; a distant branch is never automatically chosen to overwrite its history.

During a partition within one domain, only a component able to satisfy the same quorum and locking rules can continue. Minority components wait. Two components do not gain independent access to the same outputs by declaring themselves surviving communities. If a community needs its own payment availability, it must establish a separate domain and relocate its funds while the old authority is still available.

5. Export: surrender before sending

Before payment, a recipient can publish a signed payment request containing C_RLD, destination domain identity, owner key or authorization policy, principal and acceptance conditions. The payer authenticates these fields. A copied place name, a new relay address or an unsolicited key is not an authenticated invoice. An invoice can guide admission, but it cannot give a relay authority to replace the destination or change a released payment.

The payer constructs a ticket T naming the source A, destination B, exact recipient, C_RLD, principal v, source fee f_A, destination import fee f_B, recovery owner, recovery fee r, source inputs, owner nonce and provenance commitment. The stable domain identities bind authenticated initial descriptors; the source must support the destination rejection-verification rule before accepting an export that offers recovery. The destination service-fee rule and the recovery rule are also bound. All amounts are atomic RLD units. The exported amount is a = v + f_B. Source inputs cover a + f_A and any explicit change. A ticket has no elapsed-time rule that restores its value at A.

```
x = H("export", protocol, C_RLD, A, B, canonical(T))
a = v + f_B
require ticket and all inputs bind C_RLD
require v > 0, 0 <= r <= a, and bounded nonnegative amounts
require all input owners sign T and its exact change
require no input is already spent or exported
```

Export is one atomic source transition. It consumes the inputs, creates source change and the authorized source service fee, and inserts E[x] = Exported(T, a). The amount a is no longer spendable at A. The finalized export record is committed into the source state and accompanied by its inclusion proof and the source consensus commit evidence. The owner signature alone is not an export certificate.

Certificate release follows durable local finalization. A crash before finalization supplies no transfer authority. A crash after finalization must recover the same export, rather than refunding it or authorizing an alternative spend. Wallet intent and exact bytes are retained before submission so that a lost response can be reconciled against certified state. A local reservation before export is useful wallet bookkeeping; the committed export is the monetary lock.

The payer can cancel an unfinalized proposal only through the source's ordinary conflict-resolution rules. Once exported, no source timeout, sender absence, carrier acknowledgment or local administrator can make a spendable replacement. This constraint is necessary because a valid copy may already be traveling toward B.

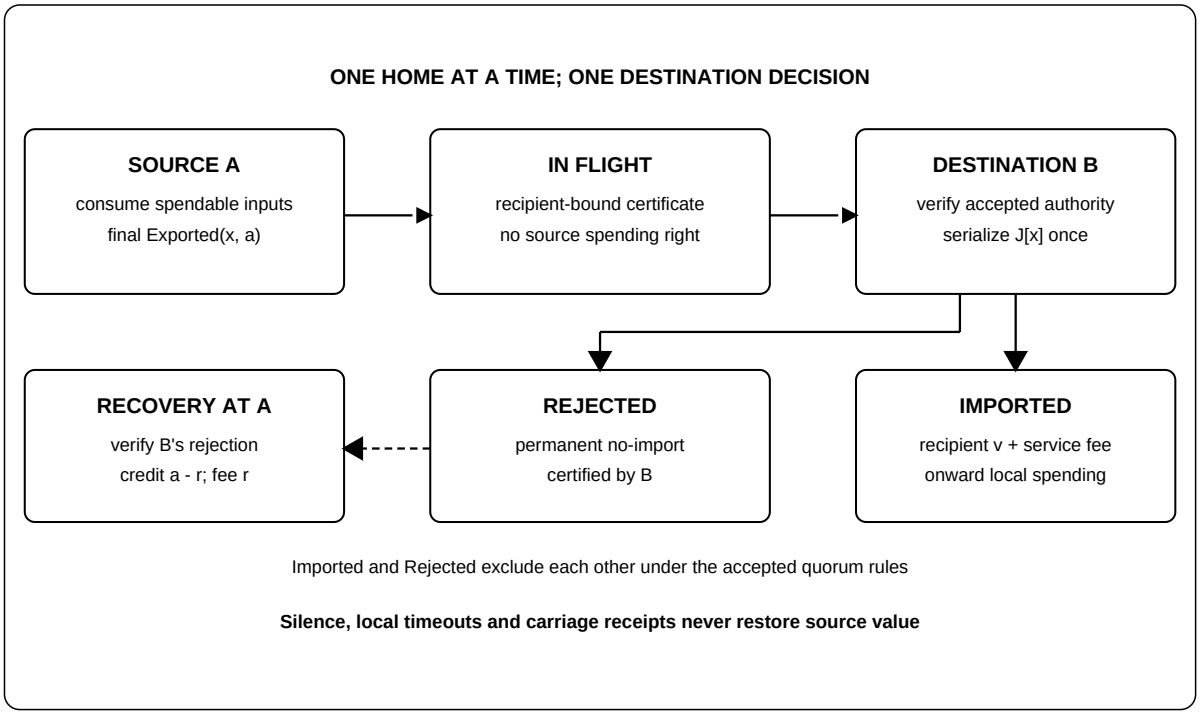


Figure 1. One exported amount can become one destination credit or one source recovery. The destination's permanent decision excludes the other outcome; a transport receipt grants neither.

6. Import: a single destination decision

The forward package contains T and its owner authorizations, source export inclusion proof, complete commit evidence, and the source configuration chain needed to reach an accepted trust anchor. The destination requires the common C_RLD and checks the exact domain identities, all bound amounts and policies, the source commit semantics and proof membership, and its own provenance and resource admission rules. An arbitrary source root or a relayer's attestation cannot initialize a trusted remote state.

The baseline verifies certified source execution under the accepted source-validator assumption. It does not claim that a Merkle path independently proves all historical transaction validity. A validity-proof alternative would add a particular proof system and data-availability assumptions; it is not silently supplied by the word certificate. Established interoperability protocols similarly distinguish remote consensus verification from transporting messages [5].

Any party may submit a complete authentic package to B; neither the payer nor recipient must be online to sign again. B processes a valid package through its own consensus. The atomic import transition requires J[x] to be absent, sets J[x] = Imported(H(T)), and creates the exact principal v for the named recipient plus f_B for the bound destination service rule. The output's home is B. These effects finalize together; a crash cannot durably credit the recipient without also consuming the ticket identity, or consume it while losing a finalized output.

```
Import(T, source_commit):
  require T.currency = C_RLD
  require authentic final Exported(T, a) at source A
  require T.destination = this stable domain B
  require J[x] is absent
  require current destination admission accepts T
  atomically:
    J[x] = Imported(H(T))
    U += recipient(v), destination_service(f_B)
```

An exact duplicate returns or reconstructs the existing result without creating value. A different ticket under the same identifier, a ticket addressed to another domain, or an object with mixed proof fragments is rejected. The decision namespace uses the full currency/protocol/domain binding, not a transport message number. Source and destination identifiers remain stable through authorized key changes, so switching epochs does not create a fresh import namespace.

After destination finalization, the recipient may spend locally or export onward. A return message to A can report the import and improve the payer's information, but is not a condition of B's credit or the recipient's next payment. If A is later unreachable, an already obtained authentic export can still be imported provided B retains a valid source trust chain and all security assumptions remain applicable.

A relay may carry A-to-B evidence through C without C becoming a monetary intermediary. Conversely, importing into C and then exporting to B is a distinct monetary route with additional fees and trust dependencies. Physical route selection cannot silently substitute the latter for the former.

7. Rejection, cancellation and stranded value

A source cannot distinguish a lost message from an imported payment whose receipt was lost. Restoring the source balance in both cases would duplicate value in the second case. The basic protocol therefore has no automatic elapsed-time refund. Transport expiry, an invoice deadline and a local user-interface timeout can stop a delivery attempt; none reverses E[x].

Recovery requires positive destination evidence. A payer-authorized rejection request carries the authentic export and the recovery-owner authorization bound by T. The recipient may also authorize rejection. The destination verifies the same source identity and export evidence used for import, and serializes rejection against import in J. If J[x] is absent, it may atomically set J[x] = Rejected(H(T)). If already Imported, it cannot issue a valid rejection. A repeated Rejected decision returns the same outcome. Any destination operation fee for requesting rejection is separately funded under its local admission rules.

Rejected is a permanent monetary decision, not a temporary delivery error. It forbids every later import of x, including after restart, state compaction and validator replacement. A malformed package, full queue or unrecognized source yields a refusal or Pending status, not a usable rejection certificate. To certify rejection, B must accept enough of the ticket's authenticated identity and configuration rules to bind a permanent decision. This requirement prevents a source from treating an unauthenticated error as a refund.

```
Recover(T, destination_rejection):
  require authentic final J[x] = Rejected(H(T)) at B
  require E[x] is exactly Exported(T, a), not Refunded
  require T's bound recovery rule permits amount a - r
  atomically:
    E[x] = Refunded(H(T))
    U += recovery_owner(a - r), source_service(r)
```

The source authenticates the destination decision through the destination's accepted initial descriptor and configuration chain, not through keys supplied as an unverified field in the rejection. A replay cannot recover twice. If policy or fee changes make processing unavailable, the operation waits or uses a separately authorized sponsor; the protocol does not alter the recipient, principal or refund formula to force progress.

Import and rejection requests can race. Their order at B determines which terminal decision exists. A payer's request is not a guaranteed recall, and a source export is not yet destination settlement. Once imported, reversal requires a new payment authorized by the current recipient. Rerouting an unimported ticket requires certified rejection and recovery followed by a new owner-authorized export; the carrier cannot edit T.

If B is permanently destroyed or cannot assemble its required quorum, the source may never obtain either import or rejection evidence. The value remains in flight and unavailable at A. This is the explicit availability cost of preserving exclusivity without remote communication. Insurance, credit and negotiated compensation may address the economic loss through separately funded arrangements. They do not change the original ticket's state or create a second claim by declaration.

8. An interstellar payment and local autonomy

Alice in domain A pays Bob in domain B. Alice verifies Bob's request, authorizes T, and waits for A's final export certificate. A removes the exported amount from its usable balance set. Alice gives complete copies to willing carriers and may go offline. Bob may be offline while those copies cross successive one-way contacts.

When the package arrives, B checks the accepted source authority and exact ticket, then finalizes its one import decision. Bob obtains a normal local output. He can pay Carol in B under B's local quorum without querying Alice or A. He can also authorize an export to a third domain, which repeats the same surrender-and-import mechanism. Bob's receipt back to Alice can follow another route years later without holding up these local payments.

```
A: Alice's spendable funds -> final export x
Physical contacts: copy and carry x plus complete evidence
B: undecided x -> final Imported(x) -> Bob's local output
B: Bob -> Carol through ordinary local settlement
Optional return: evidence of import, with no source re-credit
```

Disconnection from A cannot, by itself, reverse B's finalized payments. There is no global competition in which A's greater work replaces B's history. That property follows from preassigned conflict authority and local finality, and remains conditional on the integrity of the authorities in the funds' provenance. Reconnection brings new payments, receipts and incident evidence; it does not add two versions of a balance together.

Two disconnected domains can both continue using their distinct local funds. A partition inside B cannot create two B quorums with independent copies of those funds under the fault assumption. A ship wanting autonomous payments therefore needs its own accepted domain and transferred balances before departure. Merely running another copy of B's software does not supply a new authority or a new balance.

A ship with one validator is a domain only under a zero-validator-fault assumption. It has no Byzantine replication tolerance and depends on that validator's durable state and availability. The architecture does not label one surviving machine as equivalent to an independent fault-tolerant community. Safety-preserving emergency replacement requires the existing authorized handoff rule; inability to meet that rule can mean a permanent halt.

9. Provenance and the spread of trust

Cross-domain finality is conditional, not trust-free. If an accepted source quorum can certify fictitious exports or conflicting histories, signatures on those statements do not make the value real. A destination cannot learn an undisclosed remote compromise from the absence of an alarm. Waiting for a fixed local interval does not solve an unbounded stellar communication delay.

Each output therefore carries an authenticated provenance set P of authority descriptors and configuration histories on which its backing has depended. Genesis or issuance creates the initial set. A local transfer propagates the union of its inputs' sets. Export binds that union plus its source authority; import adds the destination authority. A recovered output and its recovery-fee output retain the original provenance and add both the source and the destination configurations that authorized rejection and recovery. Source change and source fees retain their input provenance plus the executing source

authority. Combining outputs never removes an authority from their provenance. Authorized validator succession extends the relevant configuration history rather than erasing its old fault assumptions.

Every receiving policy requires C_RLD and specifies which authority histories and configurations it accepts. Provenance is security metadata for RLD, not a domain token identifier or permission to change its denomination. Before spending or accepting, the user can inspect the resulting set. Value routed through a reputable domain does not automatically acquire that domain's trust alone. If A-imported value is passed through B, B must preserve A's provenance. Ordinary transport through a carrier adds no monetary authority; monetary import into that carrier's domain does.

This rule is enforced by honest domain validators and their authenticated state transitions. It is not a zero-knowledge proof that no hidden compromised authority ever existed. A malicious quorum that exceeds the assumed fault bound can lie about state and provenance; that is outside the conservation claim. Signatures locate responsibility and can expose equivocation once evidence meets, but do not recover missing backing or guarantee a market value.

Two RLD balances with different provenance can face different acceptance conditions or market discounts for security risk. Their protocol denomination remains identical; a shared ticker does not eliminate counterparty or validator risk. A wallet must not merge them into an apparently risk-free total. Limits on accepted principal or outstanding tickets can bound a particular wallet's declared exposure at admission, but do not bound losses across an arbitrary economy. Repeated circulation and contingent liabilities require a separate risk model. No global loss cap is inferred from a per-peer quota.

Explicit provenance makes an uncomfortable tradeoff visible: an open network can let anyone create and advertise a domain, while recipients retain the right to refuse its monetary authority. Universal automatic acceptance of arbitrary new validators would require an additional Sybil-resistant security construction. This paper does not disguise that missing assumption as permissionless consensus.

10. Transport and durable evidence

Relays provide store-carry-forward service across actual contacts [6]. The transport envelope identifies its version, exact content hash and length, origin authentication and destination domain. A fragmented object binds all piece positions to the same whole commitment. The receiver reconstructs and authenticates the complete object before monetary processing. Duplicate copies, parallel routes and repeated custody acknowledgments never create a second ticket decision.

An admitted custodian durably stores the agreed complete bytes before acknowledging custody. A digest without the material is not retained evidence. Capacity refusal occurs before accepting a new obligation; an existing custody contract specifies retention and failure reporting. Replication across independent carriers can reduce exposure to a lost copy, but correlated loss and fraudulent storage claims remain possible. A signed receipt proves a signed custody assertion, not future storage, delivery or payment finality.

End-to-end authentication survives relay changes; contact authentication protects the particular link [7]. A two-way link may negotiate missing dependencies. A one-way transfer carries its complete required export and authority material, or arranges for the missing material to arrive by another route. Sender and recipient need not be simultaneously online. Discovery helps locate candidate contacts, while a bootstrap still needs authentic rules and a first real contact or carried archive.

For distance d, propagation speed at most c implies that new information cannot arrive before d/c. Actual settlement also includes contact waiting, serialization, proof verification and destination consensus. Where a path and the necessary trust material already exist, ordinary import has no protocol-mandated interstellar round trip. Recovery after rejection requires destination evidence to travel back to the source and consequently has a different latency.

Forwarding schedules, physical media, energy and carrier motion are outside monetary consensus. A route advertisement cannot promote a new domain descriptor to a trust anchor or change T's destination. The absence of a universal Earth directory removes one service dependency; it does not create an unavailable physical route or guarantee that a carrier will accept the job.

11. Fees, liquidity and incentives

Local service fees are explicit owner-authorized RLD transfers to the payout rule committed by the domain. They pay for validation, bandwidth, durable state and administration under that domain's economic policy. There is no reward for merely generating identities or duplicating packets. The only new RLD is the fixed public-work reward in Section 3; validator operating budgets and local fee policies cannot draw additional issuance. A validator may also compete as a miner under the same work and slot rules, without a privileged payout. A low or zero fee policy needs some other declared funding source.

A successful cross-domain payment moves a + f_A out of the payer's inputs: f_A goes to source service, while a = v + f_B becomes an exported right. Import creates v for the recipient and f_B for destination service. Rejection leaves a unavailable until recovery, which creates a - r for the recovery owner and r for source service. These exact amounts and rules are authorized in T. A change in future operating costs may require a sponsor or cause delay; it cannot silently take additional principal.

Relaying is a separate purchased service, with protocol fees quoted and settled in RLD. A carrier can quote a fee for a finite custody and forwarding contract, paid locally or through separately authorized tickets. Prepayment exposes the customer to carrier failure; payment after proof of delivery exposes the carrier to delay and evidence availability. A custody receipt alone does not settle this fair-exchange problem. Honest service providers need funded contracts, reputation, collateral or legal arrangements appropriate to their risk; none is assumed free or automatically supplied by validator fees.

Transferred authority has a capital cost. If exports depart at rate lambda units per unit time and their average unavailable interval is W, an average in-flight inventory of approximately lambda*W follows under stable-flow conditions, applying the flow relation associated with Little's law [14] to units of value. Long contact gaps can therefore tie up substantial value even if each certificate is small. Prepositioning funds, reciprocal trade and liquidity providers can improve local usability, but credit and exchange introduce their own pricing, collateral and counterparty assumptions.

The basic mechanism transfers only RLD. Exchange of RLD for an external currency or delivery of physical goods is outside the native payment state machine and is not automatically atomic across stellar delay. A provider promising immediate destination liquidity is making an additional economic arrangement; it must not present that credit as a completed import of a ticket that has not arrived.

12. Conservation and safety argument

Fix the common C_RLD and a causally consistent cut of the execution: every included destination action includes its causal source export. Let U be the total spendable value, including service-fee outputs, across all domains at that cut. Let F be the principal plus reserved destination fees in exports that have not yet been imported or recovered. A permanently rejected ticket remains in F until source recovery; a copied packet contributes nothing separately. Let I be the sum of finalized strand rewards, G = 0 the initial RLD supply and Z cumulative RLD retirement. All quantities are atomic RLD units. The invariant is:

```
U + F = G + I - Z
```

F is a mathematical accounting category over the actual causal execution, not a total that a disconnected source can infer from missing receipts. After an import, a stale source may still display Exported, but its entry is not an additional spendable allocation. A domain's observable balances, pending exports and receipts must remain separate from any claim to a live global balance.

Lemma 1: one local decision. Two quorums of size 2f + 1 among 3f + 1 validators intersect in at least f + 1 validators. With at most f Byzantine, the intersection contains an honest validator. The selected consensus protocol's durable locking and commit rules prevent honest validators from participating in conflicting finalized state transitions. Quorum arithmetic alone is insufficient without those rules and valid reconfiguration. Thus, under the local safety assumptions, one input cannot be finalized into conflicting spends and J[x] cannot finalize as both Imported and Rejected.

Lemma 2: one destination. The owner authorization and source certificate bind the stable destination identity into T and x. An import transition checks that identity. Other domains cannot use the same certificate to credit themselves under the protocol. Duplicate delivery within B encounters the same permanent J[x]. The result holds across restarts and authorized epochs only if the decision map and its authenticated continuity are retained.

Lemma 3: import excludes recovery. Recovery requires a final Rejected decision from B. By Lemma 1 that decision excludes Imported; by Lemma 2 no other domain can import the same ticket. Source recovery additionally changes the exact E[x] once to Refunded. Therefore a ticket cannot produce both destination spendability and source recovery under the relevant source and destination fault assumptions. A conflicting signed import and rejection is evidence that an assumption failed, not a rule for choosing both.

Proposition: conservation. The authenticated zero-initial-supply state satisfies the invariant. A local transfer preserves U. Export decreases U by a and increases F by a; source fees only redistribute U. Import decreases F by a and increases U by v + f_B = a. Rejection changes neither U nor F. Recovery decreases F by a and increases U by (a - r) + r = a. Valid issuance and retirement change the right-hand side by the same amount as spendable or in-flight rights. Lemmas 1-3 exclude a second application of any value-creating branch. Induction over the causal execution therefore preserves the equation.

Proposition: isolation does not reorganize local settlement. A domain's valid next state extends its own finalized prefix. Import consumes a certificate; it does not replace that prefix with a remote branch. Consequently, loss or restoration of an inter-domain connection cannot by itself invalidate an already finalized local output. This is a conditional local-finality property, not a proof that a compromised provenance authority or stolen owner key cannot cause loss.

Conditional progress. Suppose the source has finalized an export, the required complete evidence eventually reaches the destination, its admission policy accepts the ticket, its correct local quorum makes progress and no authorized rejection is finalized first. Fair processing then finalizes import without any source return message. If rejection is finalized instead, complete rejection evidence eventually reaches the source and its correct quorum and bound recovery policy remain available, fair processing can finalize recovery once. Permanent disconnection, an unavailable quorum, expired cryptographic confidence or unfunded service invalidates the relevant progress premise; no universal deadline follows.

Proposition: bounded issuance. Let B be the sum of unissued budgets in active strands and Q the sum in exported strands not yet installed or recovered, evaluated over the same causally consistent execution cut. A rejected move remains in Q until recovery. Initially B = C and U = F = Q = Z = 0. Each Mint advances one strand counter, decreases B by rho and increases U by rho. A strand export moves its remaining budget from B to Q; installation or certified recovery moves it from Q to B exactly once. No such operation creates a spendable coin. The fixed identities, no-splitting rule and source/destination exclusion preserve one active or in-flight state per strand. Ordinary RLD transfer preserves U+F, and authorized retirement moves value to Z. Therefore:

```
I + B + Q = C
U + F + B + Q + Z = C
0 <= I <= C
```

The result is conditional on the same accepted authority and durable-state assumptions as payments, now including every strand's provenance. A Byzantine quorum above the assumed bound can lie about issuance state just as it can lie about an export; signatures and a printed cap do not independently prevent such fraud. B and Q are accounting categories, not instantaneous totals available to disconnected observers. Lost strands remain counted as unavailable entitlement rather than silently reassigned.

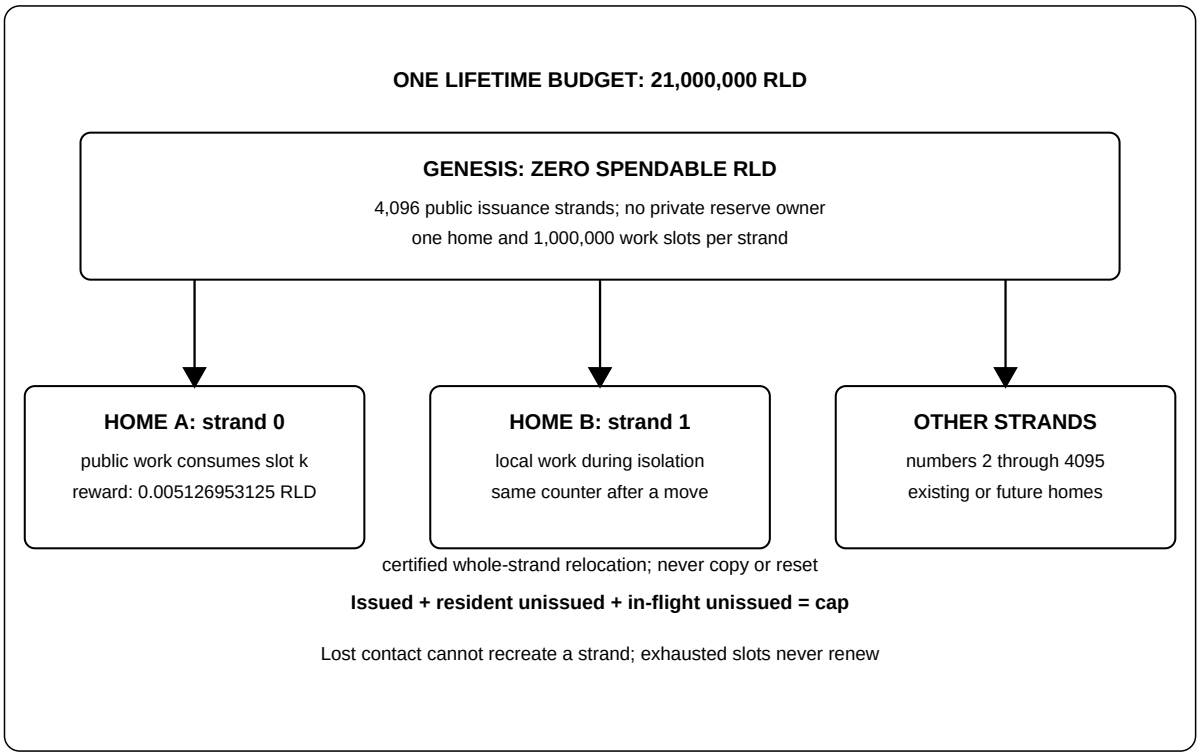


Figure 2. A fixed set of issuance strands moves between authoritative homes without copying its remaining slots. Public work converts one slot into RLD; the issued, unissued and retired accounting remains within one lifetime cap.

These arguments establish properties of the stated transition system and assumptions. They are not a machine-checked implementation proof or a proof of a new consensus algorithm. In particular, threshold compromise, lost anti-replay state, unverified configuration changes or broken cryptography invalidate the applicable premise. The concrete allocation and work rules do not establish equitable placement, economic sustainability or independent operational qualification.

13. Configuration, keys and long journeys

A domain descriptor commits to a validator-succession rule. A conservative transition finalizes a handoff containing the old and new configurations, activation position, exact predecessor, complete monetary-state commitment and import/export decision roots. It requires an old-configuration commit proof and acceptance by a new quorum. Old validators stop authorizing the old configuration beyond that boundary; new validators install and durably retain the complete state before voting. A client follows this authenticated chain from its accepted anchor. An administrator-supplied replacement root is not a valid handoff.

A handoff must preserve every unresolved payment or strand export, every Imported or Rejected decision, owner outputs, all strand counters and anchors, and complete provenance. Merely copying current balances is insufficient. If an old quorum cannot authorize succession, the baseline cannot safely declare a replacement committee and resume the same funds. A newly formed community can establish a new identity, but it cannot acquire the stopped domain's balance by naming itself its successor.

Ordinary signatures remain forgeable if the relevant old private keys are later compromised. Rotating public keys does not retroactively prevent a forged historical certificate. The baseline safety model therefore counts later compromise of an old signing quorum against that configuration's fault assumption. A stronger long-journey profile needs an explicitly instantiated forward-secure or key-evolving signature construction [13], secure erasure and independent historical anchors; none follows automatically from regular key rotation.

A traveler pins the authenticated epoch material needed for an expected journey and its stated security horizon. Long delays make revocation inherently late. A newly arrived warning can justify refusing further acceptance, but does not produce an instantaneous worldwide revocation or a funded reimbursement. Unknown suites and unsupported transitions

are rejected. Post-quantum standards can inform future profiles [8]; adopting one requires exact historical verification and renewal rules, not merely a new algorithm name.

Owner recovery is likewise an authorization policy selected before loss, such as a threshold of independently held recovery keys. Changing owners through that policy requires an ordinary certified local transition. No relay, founder or unauthenticated backup can reset an owner or recover an in-flight ticket. An old backup must reconstruct current certified state and current decisions before new signing or voting. If every recoverable copy and freshness anchor is lost, the protocol cannot manufacture the missing state.

The transparent baseline reveals amounts, public keys, provenance and some timing and route information. Encryption of carriage protects its declared audience, not all metadata. Private-payment constructions such as Zef address a different part of this problem [9]; integrating privacy would require its own proof that hidden amounts, provenance policy and duplicate prevention still compose.

14. Capacity, archives and failure bounds

Local throughput depends on validator count, execution cost, signature checks, replication, durable storage and local communication. Interstellar import adds certificate and configuration verification. Let domains i sustain completed local-payment rates r_i under declared conditions. Aggregate rate is at most the sum of those capacities after imports, exports, fees and configuration work consume their shares. That aggregate says nothing about the latency of a particular stellar route.

For a workload of one hundred million distinct payments per day, average completion rate is approximately 1157.41 per second. As arithmetic only, 64 independent domains each sustaining 25 payments per second would supply 1600 per second before overhead. This is neither a benchmark nor a guarantee that a particular quorum, device or traffic distribution reaches those rates. A highly concentrated destination remains a bottleneck. Spreading one shared balance over domains requires actual authority relocation; it is not free horizontal replication.

Permanent duplicate prevention has a permanent state cost. An authenticated map can compress a decision set into a small root for certificates, but validators still need the map data or authenticated witnesses sufficient to update it correctly. Deleting an old rejection because a ticket seems unlikely to arrive reopens the import/recovery race. Deleting an old import similarly enables duplicate credit. Snapshot renewal preserves these facts rather than declaring old packets harmless by age.

For scale intuition, 100 million new decisions daily at 64 bytes per retained entry would add about 2.34 terabytes per year before indexes, replicas and archive redundancy. A 1000-byte record per payment would represent 100 gigabytes per day before certificate overhead and replication. Domains need explicit capacity, retention funding, backup verification and recovery plans. Roots do not eliminate these resources, and these figures do not establish a concrete encoding size.

A source can snapshot finalized exports and a destination can snapshot its decision map under its authenticated local finality. New validators verify the snapshot commitment, succession and required data availability before continuing. Pruning redundant historical payloads is possible only when every needed authorization, decision and recovery witness remains recoverable under a precisely defined archive policy. A proposed accumulator or succinct proof needs its own update, soundness, availability and recovery analysis.

If a quorum, archive or accepted cryptographic profile fails, affected progress stops. Discovery and carriers may continue forwarding preserved material, but they cannot vote, issue a replacement balance or convert a storage acknowledgment into an import. Bounded admission can refuse new obligations while retaining old ones. A promise of unlimited operation on finite ordinary hardware would contradict these obligations.

15. Relation to prior work and scope of the contribution

Bitcoin established a practical open-work approach to selecting a monetary history [1]. Byzantine state-machine replication supplies conditional finality for explicitly admitted validators [2,3]. This proposal chooses the latter inside a domain to avoid making distant work competition the arbiter of an isolated community's completed transactions. That choice changes the trust and availability model; it is not an improvement in every dimension.

FastPay and Zef show that payment workloads need not pay the cost of one global total order and describe concrete authority-based payment systems [9,10]. Research on asynchronous asset transfer likewise separates transfer requirements from general consensus [11]. These results motivate locality and careful conflict scopes. Their security assumptions and measured results do not automatically transfer to this architecture.

Partitioning finite rights before disconnected use is related to escrow and bounded-counter techniques [16]. Those database results do not establish Byzantine monetary security here; the strand construction additionally relies on authenticated single-home finalization and retained decisions. IBC separates remote-consensus verification, packet transport and application transitions [5]. Cross-shard research studies atomic commit and the dangers of replay, stale messages and client-coordinated locks [12]. Surrendering source spendability and consuming a unique destination identifier belong to that established family of ideas. We do not claim to have invented a new quorum, a new cryptographic primitive or the general concept of cross-ledger transfer.

The proposed contribution is a specific interstellar payment contract combining single-home authority, local final settlement, destination-bound non-expiring exports, a permanent import-or-reject decision, evidence-dependent recovery, and explicit propagation of monetary trust. The combination is developed here for one-way contact and intervals longer than ordinary monitoring or expiry windows. Its priority and completeness require comparison beyond a limited bibliography. Interstellar terminology alone establishes no novelty.

The architecture deliberately omits a global greatest-work payment history, automatic trust in new domains, guaranteed recovery from destroyed authorities, instantaneous global revocation, arbitrary cross-domain atomic smart contracts and an unconditional cryptographic lifetime. Adding any of them changes the problem and needs additional mechanisms and assumptions. The monetary argument is about bounded RLD issuance, transfers and their composition under accepted domain histories, not every service that might be built above it.

16. Conclusion

Interstellar payment in runlai (RLD) can be organized around the location of spend authority instead of the latest globally preferred history. A community finalizes its own payments against locally homed funds. An owner paying elsewhere first surrenders local spendability, then sends a certificate bound to one destination and one recipient. The destination's permanent decision permits import or recovery, while lost messages never authorize a second allocation. Once imported, value can circulate locally without a return message to its former home.

This supports a concrete form of autonomy: interruption of distant contact does not itself overturn local settlement. It requires explicit confidence in the participating authority histories, preserved anti-replay state and sustainable local operations. Safety can leave money unavailable when a necessary authority disappears. The appropriate response is to price and disclose that risk, preposition funds where they will be used, and improve the supporting institutions and cryptography without claiming to remove causality.

A finite nonduplicable issuance budget lets prepared communities mine locally without reopening a global branch contest. It also makes placement of future issuance, stranded strands and initial authority concentration explicit. An open payment fabric built on these rules could allow communities to remain economically connected without depending on one continuously reachable center. Establishing that possibility as a useful system requires exact profiles, independent analysis and real operations. The scientific claim is narrower and testable: under the stated fault and custody assumptions, asynchronous transfer of exclusive spend authority preserves value while allowing final local payments during inter-domain isolation.

References

[1] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf

[2] M. Castro and B. Liskov. Practical Byzantine Fault Tolerance, OSDI, 1999. https://www.usenix.org/conference/osdi-99/presentation/practical-byzantine-fault-tolerance

[3] M. Yin, D. Malkhi, M. K. Reiter, G. Golan Gueta and I. Abraham. HotStuff: BFT Consensus in the Lens of Blockchain, 2019. https://arxiv.org/abs/1803.05069

[4] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA), RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032.html

[5] Inter-Blockchain Communication. How the IBC Protocol Works. https://ibcprotocol.dev/how-ibc-works

[6] S. Burleigh, K. Fall and E. Birrane. Bundle Protocol Version 7, RFC 9171, 2022. https://www.rfc-editor.org/rfc/rfc9171.html

[7] E. Birrane and K. McKeever. Bundle Protocol Security (BPSec), RFC 9172, 2022. https://www.rfc-editor.org/rfc/rfc9172.html

[8] NIST. Module-Lattice-Based Digital Signature Standard, FIPS 204, 2024. https://doi.org/10.6028/NIST.FIPS.204

[9] M. Baudet, A. Sonnino, M. Kelkar and G. Danezis. Zef: Low-latency, Scalable, Private Payments, 2022. https://arxiv.org/abs/2201.05671

[10] M. Baudet, G. Danezis and A. Sonnino. FastPay: High-Performance Byzantine Fault Tolerant Settlement, 2020. https://arxiv.org/abs/2003.11506

[11] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev and A. Tonkikh. Permissionless and Asynchronous Asset Transfer, DISC, 2021. https://doi.org/10.4230/LIPIcs.DISC.2021.28

[12] A. Sonnino, S. Bano, M. Al-Bassam and G. Danezis. Replay Attacks and Defenses Against Cross-shard Consensus in Sharded Distributed Ledgers, 2020. https://arxiv.org/abs/1901.11218

[13] M. Bellare and S. K. Miner. A Forward-Secure Digital Signature Scheme, CRYPTO, 1999. https://cseweb.ucsd.edu/~mihir/papers/fsig.pdf

[14] J. D. C. Little. A Proof for the Queuing Formula: L = lambda W, Operations Research, 1961. https://doi.org/10.1287/opre.9.3.383

[15] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications, RFC 9106, 2021. https://www.rfc-editor.org/rfc/rfc9106.html

[16] V. Balegas, D. Serra, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguica, M. Shapiro and M. Najafzadeh. Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants, 2015. https://arxiv.org/abs/1503.09052